

YOLO

■ 날짜	@2025년 9월 16일
■ 관련 과목	 <u>보아즈</u>

배경과 문제의식

YOLO (You Only Look Once) 모델이 등장하기 이전의 **객체 탐지(Object Detection)** 시스템들은 여러 단계로 이루어진 복잡한 파이프라인을 사용했음.

이 시스템들은 객체 탐지를 **분류(Classification)** 문제로 접근했기 때문에, 객체를 탐지하기 위해 이미지의 여러 위치와 다양한 크기에 걸쳐 분류기를 반복적으로 실행해야 했음.

- DPM (Deformable Parts Models)과 같은 초기 시스템은 **슬라이딩 윈도우(Sliding Window)** 방식을 사용해 이미지 전체를 훑으며 분류기를 실행했음.
- **R-CNN**과 같은 최신 시스템들은 먼저 **영역 제안(Region Proposal)** 방법을 사용해 객체가 있을 법한 잠재적인 바운딩 박스를 생성한 뒤, 이 제안된 박스들을 개별적으로 분류하는 방식을 사용함.

이러한 복잡한 파이프라인은 여러 개별 컴포넌트들을 각각 따로 학습시켜야 했기 때문에 최적화가 어렵고 속도가 매우 느리다는 근본적인 한계를 가지고 있었다.

예를 들어, R-CNN은 테스트 이미지 한 장을 처리하는 데 40초 이상이 걸리기도 함.

기본 YOLO 모델은 초당 45프레임의 속도로 이미지를 실시간 처리한다.

더 작은 버전인 Fast YOLO는 무려 초당 155프레임을 처리하면서도, 다른 실시간 탐지기보다 mAP(mean Average Precision)을 두 배나 높게 달성한다.



1) DPM (Deformable Parts Model)

- **아이디어:** 물체를 "부분(Parts)"으로 쪼개서 봄.
예를 들어 사람 → 머리, 팔, 다리 같은 부분.
- **동작 방식:** 각 부분은 **HOG(Histogram of Oriented Gradients)** 같은 전통적 특징으로 표현되고, 이 부분들이 서로 어떻게 배치될지를 "변형 가능한 구조(Deformable)"로 모델링함.
- **장점:** 사람처럼 복잡한 모양도 "부분 조합"으로 인식 가능.
- **단점:** 특징 추출과 탐지 과정이 전부 수작업 설계(HOG 등)라 성능 한계가 있었고, 속도도 느림

2) 슬라이딩 윈도우 (Sliding Window)

- **아이디어:** 이미지를 작은 창(window)으로 이리저리 훑으면서 "여기 물체 있나?" 하고 검사하는 방식.
- **과정:**
 1. 창 크기를 다르게 조절 (scale)
 2. 이미지를 위·아래·좌·우로 조금씩 움직이며 검사 (stride)
- **문제점:**
 - 엄청난 연산량 → 속도 매우 느림.
 - 창 안에 물체가 정확히 맞아떨어지지 않으면 성능 저하

슬라이딩 윈도우와 커널의 차이?

- **공통점:** 둘 다 "윈도우를 움직이며 연산"하는 구조.
- **차이:**
 - **슬라이딩 윈도우 탐지:** 분류기를 창마다 돌림. "이 영역은 고양이인가?"
 - **CNN의 커널 연산:** 필터(커널)가 픽셀 패턴을 찾으며 이동. "이 영역에 수직선 패턴 있나?"
- 즉, 커널은 **특징(feature) 추출용**, 슬라이딩 윈도우는 **물체 존재 여부 판단용**.

3) R-CNN (Region-based CNN)

- **아이디어:** 슬라이딩 윈도우처럼 전부 다 보지 말고, "물체 있을 법한 곳 (Region)"만 뽑아 검사하는 것.
- **구성 단계:**
 1. **Selective Search:** 후보 영역(region proposal) ~2000개 추출.
 2. 각 영역을 CNN에 넣어 **특징 벡터** 뽑음.
 3. SVM 분류기로 "이건 고양이/개/배경" 판별.
 4. 회귀(regression)로 박스 위치 보정.
- **장점:** 슬라이딩 윈도우보다 훨씬 정확.
- **단점:** 여전히 복잡한 파이프라인. 한 장당 40초 이상 걸릴 정도로 느림

핵심 아이디어 및 모델 구조

YOLO는 이러한 기존 방식의 문제점을 해결하기 위해 객체 탐지 문제를

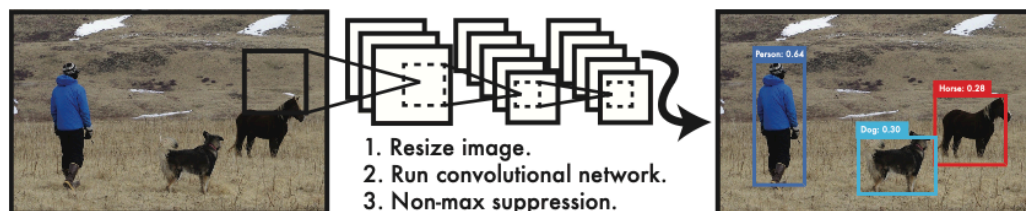
단일 회귀(이미지 → (박스+클래스)) 문제로 재해석했음.

즉, 이미지의 픽셀에서 바로 바운딩 박스 좌표와 클래스 확률을 예측하도록 했다.

이처럼 전체 탐지 파이프라인을 하나의 신경망으로 통합함으로써 "이미지를 한 번만 본다 (You Only Look Once)"는 뜻의 YOLO라는 이름이 붙었음.

→ **단 한 번의 신경망 실행으로** 위치와 클래스를 동시에 예측하자는 것.

이렇게 하면 **엔드투엔드**로 검출 성능에 맞춰 최적화할 수 있고, 속도도 획기적으로 빨라질 수 있음.



1. 모든 입력 이미지를 **448×448 정사각형**으로 바꿈.

2. 바뀐 이미지를 하나의 CNN(합성곱 신경망)에 집어넣음.

3. 이 CNN은 이미지를 구석구석 살펴서

- “여기에 자동차 있음”,
- “저기에 사람 있음”,
- “강아지 위치는 여기”

같은 **바운딩 박스 좌표 + 클래스 확률**을 한 번에 예측

4. 신뢰도에 따라 걸러냄 (Threshold by confidence)

- 신경망은 여러 개의 후보 박스를 뽑아내는데, 그중에는 “확실하지 않은 예측”도 섞여 있음.
- 각 박스마다 confidence score(이게 진짜일 확률)가 함께 나오는데, 이 값이 일정 기준(예: 0.5 이상)보다 높은 것만 “탐지 성공”으로 인정함.
- 즉, YOLO는 “자신 있는 결과만 남기고, 애매한 건 버린다”는 원리.

하나의 합성곱 신경망이 동시에 여러 바운딩 박스와 해당 박스들의 클래스 확률을 예측한다.

YOLO는 전체 이미지를 학습 데이터로 사용하고, 탐지 성능을 직접 최적화한다.

이 통합 모델은 기존 객체 탐지 방법들에 비해 여러 가지 장점을 갖는다.

YOLO는 입력 이미지를 **SxS 그리드(Grid)**로 나눔.

만약 어떤 객체의 중심이 특정 그리드 셀 안에 위치하면, 그 그리드 셀이 해당 객체 탐지를 담당하게 됨.

- **바운딩 박스 예측:** 각 그리드 셀은 **B**개의 바운딩 박스와 각각의 신뢰도 점수를 예측함.

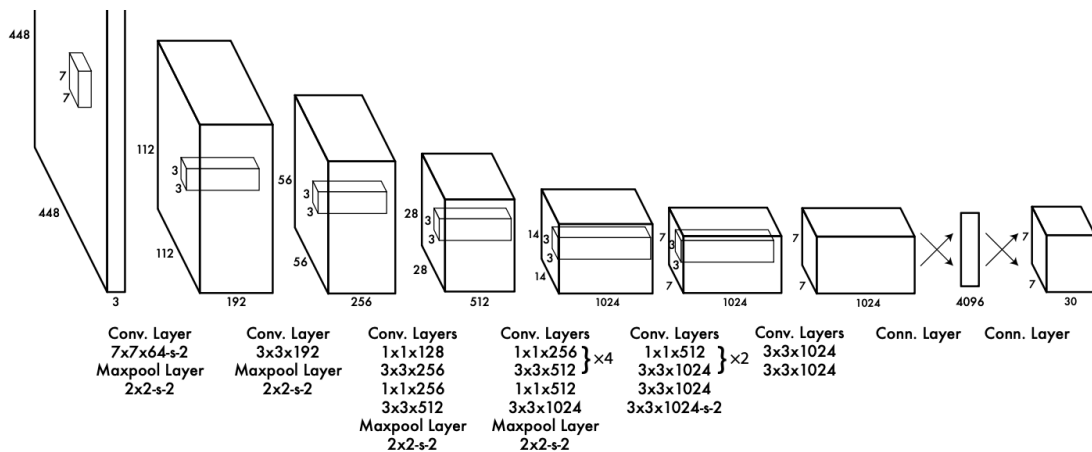
신뢰도 점수는 해당 박스가 객체를 포함할 확률과 예측된 박스의 정확도를 동시에 나타냄.

→ “이 박스에 객체가 있을 확률 × 박스가 정답과 얼마나 겹치는지(IOU)”를 의미

- **클래스 확률 예측:** 각 그리드 셀은 **C**개의 조건부 클래스 확률, 즉 그리드 셀에 객체가 있다는 조건 하에 각 클래스일 확률을 예측함.

이러한 예측값들은 **S x S x (B * 5 + C)** 크기의 텐서로 인코딩된다.

모델 구조 (Network Architecture)



YOLO의 네트워크 구조는 GoogLeNet 모델에서 영감을 받았음.

- **기본 YOLO:** 24개의 **합성곱(Convolutional) 레이어**와 2개의 **완전 연결(Fully Connected) 레이어**로 구성되어 있음. 이미지 분류를 위해 미리 학습된 합성곱 레이어를 사용하고, 여기에 4개의 합성곱 레이어와 2개의 완전 연결 레이어를 추가하여 객체 탐지용으로 미세 조정(fine-tuning)함.



→DNN의 Flatten 문제와 닮지 않았나?

- DNN의 한계에서 말했듯:

Flatten은 공간 구조(픽셀 간 위치 관계)를 **끊어버림**.

→ 그래서 작은 물체의 정밀 위치 정보가 사라지기 쉬움.

- YOLO v1도 똑같은 약점이 존재함.

- Conv로 공간적 특징을 잘 뽑아냈지만, 마지막에 Flatten → FC로 연결하면서

격자 단위 위치 정보는 살리지만, 더 정밀한 공간 구조는 손실됐음.

- **Fast YOLO:** 더 빠른 속도를 위해 합성곱 레이어 수를 9개로 줄이고 필터 수도 줄인 경량화된 버전.

→ 네트워크 크기를 줄인 것 외에는 학습·테스트 파라미터는 YOLO와 동일함.

YOLO는 이미지 전체를 한 번에 처리하여 바운딩 박스와 클래스 확률을 예측하므로, 기존 방식보다 훨씬 빠르다. 기본 YOLO는 초당 45프레임을, Fast YOLO는 초당 155프레임을 처리할 수 있어 **실시간 성능**을 자랑함.

